Thí nghiệm 2: DM tự sinh tuyến cho nhiều luồng khẩn cấp trong bão Yagi

Hướng 1 + 3: bỏ giới hạn 8 tuyến ứng viên, giảm chi phí DM — kết quả có tiêu chí chốt trước (TV2, 20/10/2026)

Tóm tắt. Theo đúng 5 tiêu chí đã chốt, **DM thắng 0/9 ô** (3 quy mô × 3 mức tải). DM tự sinh tuyến tốt hơn Dijkstra tuần tự có ý nghĩa thống kê ở tải nhẹ và vừa (6/9 ô) và cách mốc dưới 0,35% ở các ô đó; nhưng ở tải nặng nó cách mốc 2,2–5,0%, không hơn được Dijkstra tuần tự hay lời giải trong 8 ứng viên, và luôn kém tìm kiếm cục bộ không giới hạn chạy 1 lần (TKCB-KGH 1 lần) — phương pháp này vừa nhanh hơn vừa tốt hơn DM ở cả 9 ô. Điểm tích cực duy nhất đúng như dự đoán: **thời gian DM gần như không tăng theo số luồng** (42 → 54 → 65 ms khi số luồng tăng từ 30 lên 80 rồi 160), trong khi mốc tăng gần tỉ lệ thuận (57 → 168 → 337 ms); tỉ lệ thời gian DM / mốc giảm từ 0,74 xuống 0,19, nhưng chưa tới ngưỡng 0,1 của C4.

1. Câu hỏi, thiết kế và tiêu chí

Thí nghiệm 1 cho thấy hai điểm nghẽn: (a) mọi phương pháp chỉ được chọn trong 8 tuyến ứng viên tính sẵn khi chưa có tải; (b) thời gian DM tăng theo số luồng. Thí nghiệm 2 giữ **đúng các tình huống của thí nghiệm 1** (cùng 16 kịch bản bão Yagi, cùng luồng, cùng ba mức tải, ba quy mô 20–40 / 80 / 160 luồng) nhưng cho mỗi luồng đi *bất kỳ* đường hợp lệ nào. DM mới làm việc theo *cặp nguồn – đích* (mỗi tình huống chỉ 5–12 cặp, dù có bao nhiêu luồng): nó khử nhiễu một "bản đồ cạnh nên dùng" cho mọi cặp cùng lúc; tuyến được dựng từ bản đồ đó. Mô hình chỉ huấn luyện ở 20–40 luồng, trên 600 kịch bản bão / mưa lớn khác (loại toàn bộ 04–12/9/2024).

Mốc (lời giải tốt nhất tìm được) là **TKCB-KGH 10 lần**: tìm kiếm cục bộ không giới hạn tuyến — lần lượt từng luồng đổi sang đường Dijkstra theo chi phí biên — từ 10 điểm xuất phát. Tiêu chí ghi trong `results/yagi2/TIEU_CHI_2.md` trước khi chạy; DM thắng một ô nếu đạt cả năm: C1 tốt hơn Dijkstra tuần tự, C2 tốt hơn *Một-lần* (cùng mạng, đoán bản đồ một lần), C5 tốt hơn lời giải tốt nhất trong 8 ứng viên của thí nghiệm 1 — ba tiêu chí này cần khoảng tin cậy 95% của hiệu (bootstrap theo 16 kịch bản) nằm dưới 0; C3 cách mốc $\leq 2\%$; C4 thời gian trung vị $\leq 1/10$ của mốc.

Những thay đổi trước khi chấm Yagi (đều ghi trong tệp tiêu chí kèm lý do và số liệu val):

- Số mẫu S được chọn trên val trong $\{1, 2, 4\}$: mỗi lần dựng tuyến tốn bằng một lần Dijkstra tuần tự ($\approx$ 1/27 thời gian mốc), nên giữ $S = 4$ thì C4 không thể đạt.
- **Đổi cách dựng tuyến.** Cách đã chốt (Dijkstra tuần tự cộng điểm phạt theo bản đồ) kém hơn Dijkstra tuần tự thường ngay cả khi cho nó bản đồ *đúng* lấy từ nhãn (1,37% → 1,68–3,31% trên val). Thay bằng *phân rã luồng*: tách bản đồ của mỗi cặp thành tối đa 6 đường, rồi chia luồng theo tỉ lệ (pr) hoặc mỗi luồng chọn đường có chi phí biên nhỏ nhất (gan). Với bản đồ đúng: pr cách mốc 0,021%, gan 0,91%.
- Mô hình thử 5 epoch được công khai; huấn luyện lại từ đầu với H = 64, chọn checkpoint theo loss val.

Chọn trên val: không cấu hình nào đạt hạn thời gian 1/10 (nhanh nhất 8,1 ms, hạn 5,3 ms), nên theo quy tắc đã chốt chọn cấu hình tốt nhất: **DM:** $S = 4$**, 5 bước, dựng tuyến kiểu gan** (val: cách mốc 2,26%); Một-lần: $S = 4$, gan (2,90%).

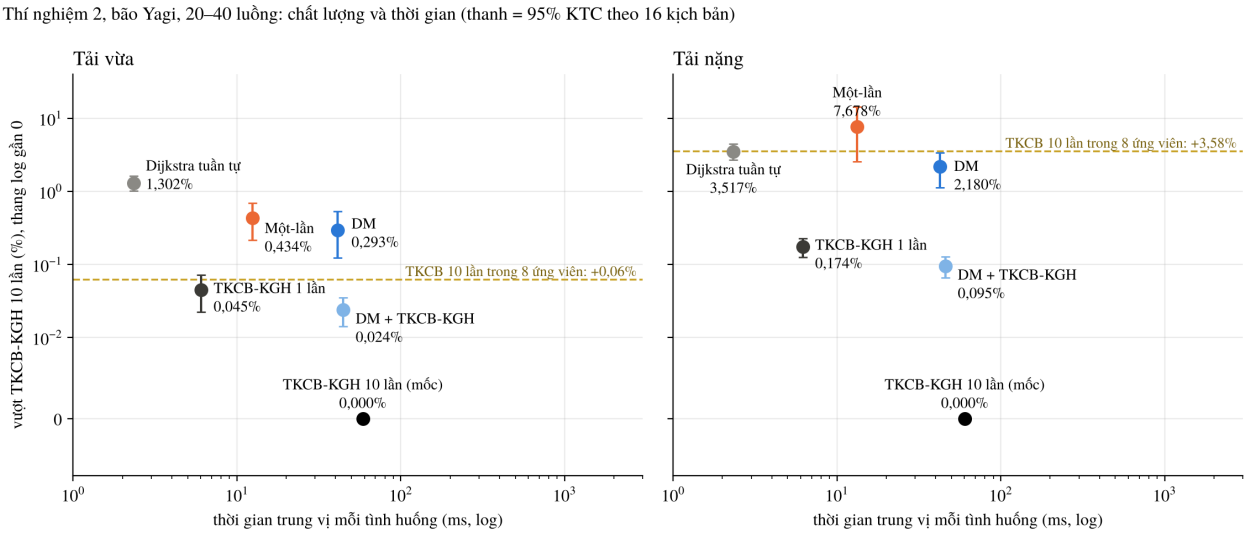
2. Kết quả: 20–40 luồng

Đọc kết quả.

- **Bỏ giới hạn ứng viên có lợi, nhưng ít hơn dự kiến trên Yagi.** Ở tải nặng, mốc không giới hạn rẻ hơn lời giải trong 8 ứng viên 3,6% (trên val là 48,8%). Ở tải nhẹ và vừa, giới hạn ứng viên gần như không gây thiệt (0,001%, 0,063%), nên DM tự sinh tuyến (0,038%, 0,293%) không thể hơn được — C5 trượt.
- **DM hơn Dijkstra tuần tự ở tải nhẹ và vừa** (tốt hơn ở 220/470 và 280/470 tình huống, kém ở 7 và 5). Ở tải nặng DM tốt hơn ở 286 tình huống, kém ở 18, nhưng vài tình huống kém rất nhiều nên hiệu trung bình

Bảng 1. Chi phí vượt mốc TKCB-KGH 10 lần (trung bình 470 tình huống mỗi mức tải) và thời gian trung vị (1 luồng CPU).

Phương pháp	Tải nhẹ	Tải vừa	Tải nặng	Thời gian
Dijkstra độc lập	+0,597%	+7,016%	+252,9%	–
Dijkstra tuần tự	+0,176%	+1,302%	+3,517%	2,3 ms
TKCB 10 lần trong 8 ứng viên (TN 1)	+0,001%	+0,063%	+3,579%	≈ 45 ms (TN 1)
Một-lần (cùng mạng)	+0,043%	+0,434%	+7,678%	12,8 ms
DM	**+0,038%**	**+0,293%**	**+2,180%**	42,1 ms
TKCB-KGH 1 lần	+0,003%	+0,045%	+0,174%	5,9 ms
DM + TKCB-KGH (báo thêm)	+0,001%	+0,024%	+0,095%	45,3 ms
TKCB-KGH 10 lần (mốc): chi phí tb	6,80 ms	7,99 ms	30,82 ms	56,9 ms
Cận dưới (chia được luồng)	−0,002%	−0,018%	−0,136%	



Hình 1. Chất lượng – thời gian ở tải vừa và nặng. Càng gần góc dưới bên trái càng tốt. TKCB-KGH 1 lần (đen) vừa nhanh hơn vừa tốt hơn DM (xanh đậm). Đường vàng: lời giải tốt nhất trong 8 ứng viên của thí nghiệm 1.

−0,11 ms [−0,29; +0,05] không có ý nghĩa.

- **DM hơn Một-lần chỉ ở tải nặng** (−0,61 ms [−1,29; −0,13]): khử nhiễu nhiều bước có ích khi các cặp tranh nhau dung lượng, đúng như thí nghiệm 1.
- **Tìm kiếm cục bộ không giới hạn lại thắng**: 1 lần chạy mất 6 ms và cách mốc 0,003–0,17%; DM mất 42 ms và cách mốc 0,04–2,2%. Dùng DM làm điểm xuất phát cho tìm kiếm cục bộ (DM + TKCB-KGH) cho lời giải tốt hơn TKCB-KGH 1 lần ở tải nặng (0,095% so với 0,174%) nhưng tốn 45 ms.

3. Quy mô: 80 và 160 luồng

- **Dự đoán về thời gian đúng một nửa.** Thời gian DM tăng chậm hơn nhiều so với tìm kiếm cục bộ (×1,5 so với ×5,9 khi số luồng tăng từ 30 lên 160), vì mạng chạy theo cặp nguồn – đích. Nhưng tỉ lệ DM / mốc mới giảm tới 0,19, chưa đạt 0,1; và TKCB-KGH 1 lần (34 ms ở 160 luồng) vẫn nhanh hơn DM (65 ms).
- **Chất lượng ở tải nặng giảm** khi số luồng tăng (2,2% → 3,4% → 5,0%), nên C3 trượt ở tải nặng mọi quy mô. Ở tải nhẹ và vừa, DM vẫn cách mốc dưới 0,35% và vẫn hơn Dijkstra tuần tự.

Bảng 2. Năm tiêu chí ở cả 9 ô. Hiệu tính bằng ms (âm = DM tốt hơn) kèm khoảng tin cậy 95%. ✓ đạt, ✗ không đạt.

Luồng	Tải	C1: DM − tuần tự	C2: DM − Một-lần	C3	C4: DM / mốc	C5: DM − 8 ứng viên	Thắng
20–40	Nhẹ	−0,011 [−0,015; −0,008] ✓	−0,000 [−0,002; 0,002] ✗	0,04% ✓	42/52 ms ✗	0,004 [0,001; 0,008] ✗	✗
	Vừa	−0,075 [−0,094; −0,055] ✓	−0,016 [−0,046; 0,013] ✗	0,29% ✓	41/59 ms ✗	0,019 [0,002; 0,041] ✗	✗
	Nặng	−0,111 [−0,285; 0,049] ✗	−0,611 [−1,290; −0,132] ✓	2,18% ✗	43/60 ms ✗	−0,123 [−0,437; 0,110] ✗	✗
80	Nhẹ	−0,013 [−0,017; −0,009] ✓	−0,000 [−0,002; 0,001] ✗	0,02% ✓	53/151 ms ✗	0,002 [0,001; 0,003] ✗	✗
	Vừa	−0,077 [−0,097; −0,058] ✓	−0,007 [−0,023; 0,010] ✗	0,34% ✓	54/171 ms ✗	0,020 [−0,000; 0,042] ✗	✗
	Nặng	−0,012 [−0,321; 0,380] ✗	−0,431 [−1,049; 0,007] ✗	3,44% ✗	54/182 ms ✗	−0,034 [−0,634; 0,547] ✗	✗
160	Nhẹ	−0,013 [−0,017; −0,010] ✓	−0,003 [−0,007; −0,000] ✓	0,02% ✓	65/294 ms ✗	0,001 [0,000; 0,003] ✗	✗
	Vừa	−0,091 [−0,111; −0,073] ✓	−0,034 [−0,060; −0,011] ✓	0,24% ✓	64/346 ms ✗	0,011 [−0,006; 0,024] ✗	✗
	Nặng	0,023 [−0,279; 0,398] ✗	−0,614 [−1,138; −0,259] ✓	4,98% ✗	66/373 ms ✗	−0,051 [−0,642; 0,509] ✗	✗

Bảng 3. Thời gian và chất lượng theo số luồng (mô hình chỉ huấn luyện ở 20–40 luồng).

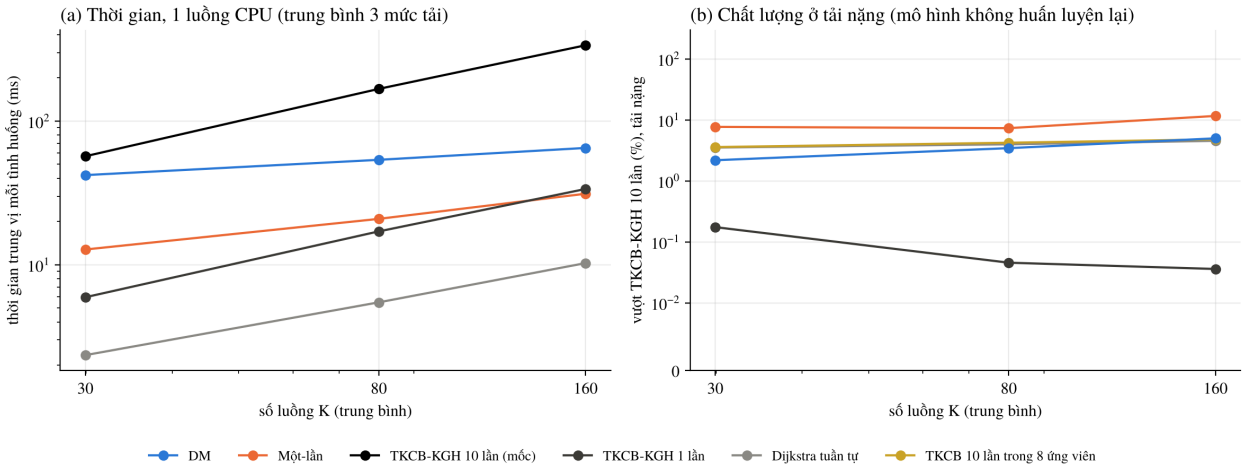
	Thời gian trung vị (ms, tb 3 mức tải)					Vượt mốc ở tải nặng	
Luồng	Tuần tự	Một-lần	DM	TKCB-KGH 1	TKCB-KGH 10	DM	TKCB-KGH 1
20–40	2,3	12,8	42,1	5,9	56,9	2,18%	0,174%
80	5,5	20,9	53,8	17,1	167,7	3,44%	0,046%
160	10,3	31,2	65,1	33,7	337,4	4,98%	0,036%

4. Kết luận và hướng tiếp

Sau hai thí nghiệm có tiêu chí chốt trước, kết luận nhất quán: **với bài toán định tuyến nhiều luồng có trễ theo tải, tìm kiếm cục bộ đơn giản đã gần tối ưu và nhanh**, nên DM — dù chọn trong ứng viên (thí nghiệm 1) hay tự sinh tuyến (thí nghiệm 2) — chưa có chỗ đứng về thực dụng. DM có hai ưu điểm thật, đã kiểm chứng: (1) khử nhiễu đồng thời tốt hơn mô hình đoán một lần cùng kiến trúc khi các luồng tranh dung lượng; (2) khi mạng chạy theo cặp nguồn – đích, thời gian gần như không phụ thuộc số luồng. Ưu điểm (2) chỉ có giá trị thực tế ở quy mô lớn hơn nhiều (hàng nghìn luồng) — chưa kiểm tra, không được suy ra từ kết quả này.

Gợi ý cho báo cáo đề tài. Nên trình bày hai thí nghiệm như một nghiên cứu *âm tính có giá trị*: thiết kế chặt chẽ (tiêu chí chốt trước, sự kiện thật giữ riêng, đối chứng mạnh), kết quả trung thực, và chỉ ra chính xác khi nào DM có ích. Nếu muốn tìm điều kiện để DM thắng, hướng có cơ sở nhất là bài toán mà tìm kiếm cục bộ yếu: ràng buộc cứng (ví dụ mỗi luồng phải có tuyến dự phòng tách rời, giới hạn số chặng) hoặc mục tiêu không tách được theo từng luồng.

Hạn chế. Một seed mỗi mô hình; CPU 1 luồng; số liệu từ bộ mô phỏng (mưa ERA5 thật); cách dựng tuyến và vài chi tiết huấn luyện được sửa sau khi xem kết quả trên val (đều trước khi chấm Yagi và đã ghi lại); hạn thời gian của C4 so với một cài đặt Python/scipy của tìm kiếm cục bộ.



Hình 2. (a) Thời gian theo số luồng: DM gần như nằm ngang, các phương pháp tìm kiếm tăng gần tỉ lệ thuận. (b) Ở tải nặng, DM kém dần khi số luồng vượt xa lúc huấn luyện, còn TKCB-KGH 1 lần tốt lên.

5. Tệp và cách chạy lại

`src/nhieu_luong/khong_gioi_han.py`	bài toán không giới hạn tuyến, Dijkstra (scipy), TKCB-KGH, phân rã luồng
`src/nhieu_luong/tao_du_lieu_kgh.py`	sinh dữ liệu (`--tap val|train|test`), cùng tình huống với thí nghiệm 1
`src/nhieu_luong/mo_hinh_kgh.py`, `huan_luyen_kgh.py`	mạng cặp – cạnh, DM, Một-lần, huấn luyện
`src/nhieu_luong/danh_gia_kgh.py`	chọn cấu hình trên val (`chon`), chấm Yagi (`test`)
`test/phan_tich_yagi2.py`	thống kê theo tiêu chí, hình
`results/yagi2/`	`TIEU_CHI_2.md`, `chon_cau_hinh.json`, `phan_tich.json`, mô hình, kết quả từng tình huống

Nhánh git `yagi-nhieu-luong` dựng trên `check_reusult` của repo nhóm (chỉ trên máy, chưa push).